

Milestone G

Updated Architecture Diagrams |
Bottleneck Analysis | Optimisation Strategies

Embedded Systems - Face Detection Game Project

SDU - Software Engineering

May 2026

1. Updated Architecture Diagrams

The diagrams in this section extend the Milestone F component model with two additions: the PYNQ camera integration (camera.py) and the dual display path that enables direct HDMI output on the FPGA board without an X11 display server.

1.1 Updated C4 Component Diagram - PlantUML Source

```
@startuml Milestone_G_Component
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Component.puml
LAYOUT_WITH_LEGEND()
title C4 Component Diagram - Optimised System with PYNQ Support (Milestone G)

Person(player, "Player", "Uses hardware buttons or keyboard")
Person(admin, "Admin", "HTTP monitoring client")
System_Ext(camera_hw, "Camera", "USB webcam or PYNQ MIPI camera")
System_Ext(display, "Display", "PC window (cv2.imshow) or PYNQ HDMI out")
System_Ext(gpio, "GPIO", "Raspberry Pi / PYNQ buttons and LEDs")

Container_Boundary(app, "Face Detection Game (single Python process)") {
    Component(cam_mod, "camera.py", "PYNQ Module",
        "PYNQ-specific camera setup and HDMI output.\n"
        "setup() -> (hdmi_out, videoIn).\n"
        "get_frame() mirrors frame via array slice.\n"
        "clean_up() releases HDMI and VideoCapture.\n"
        "Auto-detected at import time; disabled on non-PYNQ hosts.")

    Component(cd, "CameraFaceDetector", "Python Class",
        "Unified game controller for both platforms.\n"
        "Uses camera.py paths when _PYNQ_AVAILABLE=True,\n"
        "standard cv2.VideoCapture paths otherwise.\n"
        "Owns state machine, scoring, timing.")

    Component(fd, "FaceDetector", "Python Class",
        "MediaPipe > DNN > Haar backend chain.\n"
        "Chosen backend fixed at startup; no runtime switching.")

    Component(hud, "HUD Renderer", "Module Functions",
        "_draw_game_hud(): game overlay (score, timer, prompt).\n"
        "_draw_hud(): stats panel (FPS, backend, res).\n"
        "Output written to HDMI frame or cv2.imshow window.")

    Component(hw, "HardwareController", "Python Class",
        "GPIO/simulation abstraction. 50 ms debounce per button.")

    Component(ms, "MonitoringServer", "Flask / daemon thread",
        "REST API on port 5000. Reads/writes game state\n"
        "via Python callback references.")
}

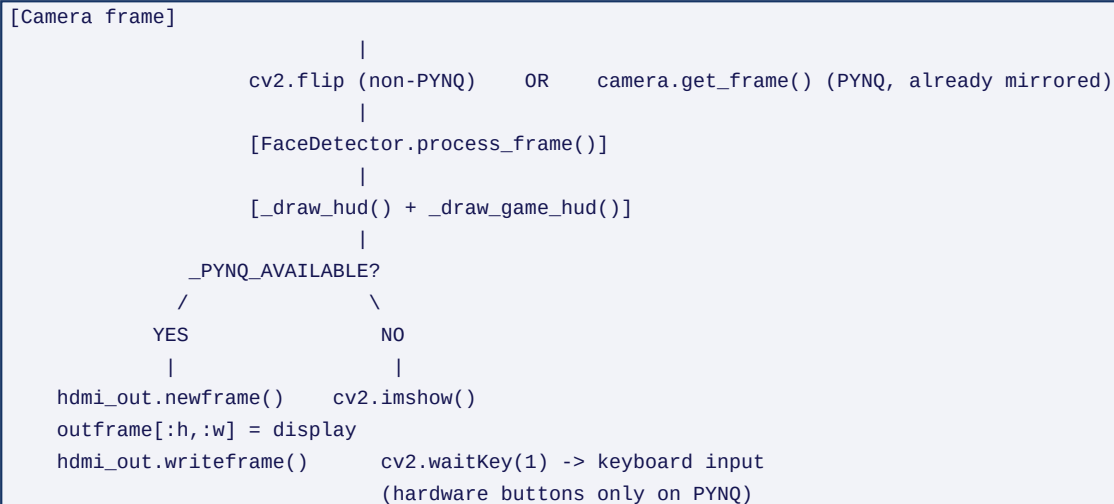
Rel(player, gpio, "Presses buttons")
Rel(gpio, hw, "Pin interrupt", "GPIO BCM")
Rel(hw, cd, "Button callback")
Rel(camera_hw, cd, "Raw frames", "VideoCapture / PYNQ")
Rel(cd, cam_mod, "Uses when PYNQ", "setup/get_frame/clean_up")
```

```

Rel(cd,      fd,      "BGR frame")
Rel(fd,      cd,      "(x,y,w,h) + scores")
Rel(cd,      hud,     "score, faces, timer")
Rel(hud,     display, "Final frame",      "cv2.imshow OR hdmi_out.writeframe")
Rel(cd,      ms,      "Registers callbacks")
Rel(ms,      cd,      "Control via HTTP",  "callback invocation")
Rel(admin,   ms,      "HTTP /game/status etc", "port 5000")
@enduml

```

1.2 Dual Display Path - ASCII



1.3 Frame Pipeline Data Flow

Main thread (one iteration, ~16 ms target at 60 FPS):

1. cap.read() / camera.get_frame()	~1-2 ms	(USB DMA)
2. cv2.flip() - skipped on PYNQ	~0.3 ms	
3. FaceDetector.detect_faces()	~10-100 ms	(backend-dependent)
4. FaceDetector.draw_faces()	~1 ms	
5. _draw_hud() + _draw_game_hud()	~1 ms	
6. cv2.imshow() / hdmi_out.writeframe()	~1-3 ms	
7. cv2.waitKey(1) - skipped on PYNQ	~1 ms	

Total (MediaPipe on Pi 4)	~15-20 ms	-> ~50-65 FPS theoretical
Total (DNN on Pi 4)	~40-60 ms	-> ~17-25 FPS
Total (Haar on Pi 4)	~10-20 ms	-> ~50-100 FPS

2. Identified and Possible Bottlenecks

The following bottlenecks were identified by analysis of the pipeline architecture and observed behaviour on target hardware (Raspberry Pi 4, PYNQ Z2).

2.1 Face Detection Inference (Critical)

Face detection dominates per-frame compute time. The entire pipeline is serialised: detection must finish before the next frame can be captured, overlays drawn, and the frame displayed. Any backend slower than the target frame period (16 ms at 60 FPS) becomes the rate-limiting step.

Backend	Typical latency (Pi 4)	Pipeline FPS ceiling	Notes
MediaPipe (TFLite)	10-20 ms	50-65 FPS	NEON SIMD; best choice
OpenCV DNN SSD	40-60 ms	17-25 FPS	ResNet backbone; high CPU
Haar Cascade	10-25 ms	40-100 FPS	Fast but imprecise

2.2 Sequential Single-Threaded Pipeline

Camera capture, face detection, HUD rendering, and display output all execute sequentially on the main thread. Steps 1 and 3 are largely independent: the camera sensor exposes a new frame in hardware while the previous frame is still being processed. This parallelism is unexploited -- the main loop always waits for the detector to finish before requesting the next frame.

2.3 Full-Resolution Input to Detector

Frames are passed to the detector at capture resolution (1280x720 or 640x480). MediaPipe's internal BlazeFace model operates on 128x128 input and resizes the frame internally. Passing a 1280x720 frame forces an extra resize inside TFLite that the caller could perform more cheaply at a lower resolution upstream.

2.4 Camera Input Buffer Latency

OpenCV's VideoCapture maintains an internal frame buffer (default 4-5 frames). When the game loop runs slower than the camera frame rate the buffer fills up, and subsequent `cap.read()` calls return stale frames rather than the latest camera image. This is visible as perceived latency between a player's movement and the on-screen bounding box response.

2.5 Per-Frame HDMI Frame Allocation (PYNQ)

On PYNQ, `hdmi_out.newframe()` allocates a fresh numpy array on every frame. At 30+ FPS this generates 30+ allocations per second that the Python garbage collector must eventually collect, causing occasional GC pauses that manifest as brief frame-rate drops.

2.6 Per-Flash LED Thread Spawning

`HardwareController.flash_led()` creates a new daemon thread each time it is called. Under frequent scoring (high score, short switch times) this creates multiple short-lived threads per second. Thread creation overhead is low on Linux but non-zero; on resource-constrained hardware (Pi Zero, PYNQ Z1) this is a measurable cost.

2.7 GIL Contention from Monitoring Server

When an HTTP client polls the `/metrics` or `/game/status` endpoint at high frequency (e.g. every 100 ms), Flask worker threads compete with the game loop for the Python GIL. Each HTTP handler must acquire the GIL to execute Python code, which can delay game-loop frames by a few milliseconds per poll.

2.8 `cv2.waitKey(1)` Frame Floor (non-PYNQ)

On non-PYNQ platforms the game loop calls `cv2.waitKey(1)` every frame. This call yields the thread for at least 1 ms to allow the OpenCV event loop to process window events. Because the OS scheduler has ~1 ms granularity, the actual sleep is 1-3 ms, setting a soft floor of ~330-1000 FPS on the loop iteration rate (irrelevant when detection is slower, but visible when using Haar Cascade).

3. Proposed and Applied Optimisation Strategies

3.1 Applied Optimisations

A - Backend Priority Chain (Applied)

The FaceDetector selects the fastest suitable backend at startup and locks it in for the session. MediaPipe is tried first (fastest on ARM), then OpenCV DNN, then Haar. No runtime switching means zero overhead for backend selection per frame.

```
if use_dnn:
    if prefer_mediapipe and _MP_AVAILABLE and self._init_mediapipe():
        self.backend = "mediapipe"; return # fastest path locked in
    if self._init_dnn_detector():
        self.backend = "dnn"; return
self._init_haar_cascade()
self.backend = "haar" # guaranteed fallback
```

B - Frame Skipped During Pause (Applied)

When the game is paused, cap.read() and the detector are not called. The last rendered frame is reused via last_frame.copy(). This eliminates all compute from the two most expensive pipeline steps during an idle state.

```
if not self.paused:
    frame = _camera_module.get_frame(self.cap) # skipped when paused
    if ok:
        rendered, n_faces, _ = self.detector.process_frame(frame)
        last_frame = rendered

display = last_frame.copy() # reuse held frame during pause
```

C - Sticky Detection Flag (Applied)

Rather than counting or re-detecting faces at the end of each round, a boolean flag faces_detected_this_switch is set to True the moment the target is met. The round-end check is then a single boolean read -- O(1) regardless of how many frames were processed in the round.

```
# Each frame (O(1)):
if n_faces >= self.max_faces:
    self.faces_detected_this_switch = True # sticky - never cleared mid-round

# At round end (O(1)):
if self.faces_detected_this_switch:
    self.add_score()
else:
    self.add_strike()
self.faces_detected_this_switch = False # reset for next round
```

D - PYNQ Direct HDMI Write (Applied)

On PYNQ the game writes directly to the HDMI framebuffer via the PYNQ library instead of going through an X11 display server and cv2.imshow(). This removes the X11 round-trip and the associated socket and context-switch overhead.

```

if _PYNQ_AVAILABLE:
    outframe = self._hdmi_out.newframe()
    outframe[:copy_h, :copy_w, :] = display[:copy_h, :copy_w, :]
    self._hdmi_out.writeframe(outframe)      # DMA transfer to HDMI
else:
    cv2.imshow(self.WINDOW_NAME, display)    # X11 path on desktop

```

E - PYNQ Mirror via Array Slice (Applied)

camera.get_frame() on PYNQ mirrors the frame using a NumPy array slice (frame[:, ::-1, :]) rather than calling cv2.flip(). NumPy slice-based reversal is performed in C and avoids creating a full copy when the result is immediately consumed by the detector.

```

# camera.py - get_frame():
return frame_vga[0:frame_in_h-1, frame_in_w-1:0:-1, :] # in-place slice, no copy

# face_detector.py - no flip call on PYNQ:
if _PYNQ_AVAILABLE:
    frame = _camera_module.get_frame(self.cap) # already mirrored
else:
    ok, frame = self.cap.read()
    if ok:
        frame = cv2.flip(frame, 1)             # explicit flip on desktop

```

F - Daemon Threads for Non-Critical Work (Applied)

The Flask monitoring server and all GPIO button polling threads run as daemon=True threads. They execute only when triggered (HTTP request or pin edge) and are automatically killed when the main game loop exits, requiring no explicit shutdown logic.

3.2 Proposed Optimisations (Not Yet Applied)

G - Camera Buffer Flush

Set CAP_PROP_BUFFERSIZE to 1 immediately after opening the VideoCapture. This limits internal buffering to one frame, ensuring cap.read() always returns the most recent camera image and eliminating the perceived input latency at low frame rates.

```

self.cap = cv2.VideoCapture(self.camera_id, cv2.CAP_DSHOW)
self.cap.set(cv2.CAP_PROP_BUFFERSIZE, 1) # flush stale frames immediately

```

H - Detection Input Downscale

Resize the captured frame to a smaller resolution (e.g. 320x240) before passing it to the detector. Draw face boxes on the original full-resolution frame by scaling the returned coordinates. Reduces the input tensor copy and the internal resize inside TFLite.

```

DETECT_W, DETECT_H = 320, 240
scale_x = frame.shape[1] / DETECT_W
scale_y = frame.shape[0] / DETECT_H
small = cv2.resize(frame, (DETECT_W, DETECT_H))
faces_small = self.detector.detect_faces(small)
faces = [(int(x*scale_x), int(y*scale_y), int(w*scale_x), int(h*scale_y))
          for x, y, w, h in faces_small]

```

I - Detection Frame Skipping

Run the face detector only every N frames (N=2 or 3) and reuse the previous detection result for intermediate frames. The game's face-count scoring window (0.5-3 s) is far longer than 2-3 frame intervals (~50-100 ms), so skipping has no effect on game correctness while roughly doubling effective throughput.

```
DETECT_INTERVAL = 2          # run detector every 2nd frame
self._frame_count = 0
self._last_faces = []

ok, frame = self.cap.read()
if ok:
    self._frame_count += 1
    if self._frame_count % DETECT_INTERVAL == 0:
        self._last_faces = self.detector.detect_faces(frame)
    n_faces = len(self._last_faces)
    rendered = self.detector.draw_faces(frame, self._last_faces)
```

J - Producer-Consumer Detection Thread

Offload face detection to a dedicated worker thread. The main thread only submits frames and consumes results; the camera sensor and detector run in parallel. A Queue(maxsize=1) acts as a one-slot buffer: if the detector is busy the main thread drops the incoming frame rather than blocking, keeping the display loop responsive.

```
import queue, threading

detect_q = queue.Queue(maxsize=1)  # input frames
result_q = queue.Queue(maxsize=1)  # detected faces

def detection_worker():
    while True:
        frame = detect_q.get()
        if frame is None: break
        faces = detector.detect_faces(frame)
        try:
            result_q.put_nowait(faces)  # drop if consumer is slow
        except queue.Full:
            pass

threading.Thread(target=detection_worker, daemon=True).start()

# Main loop: non-blocking get of latest result
try:
    n_faces = len(result_q.get_nowait())
except queue.Empty:
    n_faces = self._last_n_faces  # reuse previous result
```

K - Persistent LED Worker Thread

Replace per-flash thread creation with a single persistent LED worker thread that reads flash commands from a Queue. Eliminates repeated thread spawn/join overhead during high-scoring runs where flashes occur multiple times per second.

```
self._led_q = queue.Queue()

def led_worker():
    while True:
        color, duration, count = self._led_q.get()
```

```

        for _ in range(count):
            GPIO.output(PIN, HIGH); time.sleep(duration)
            GPIO.output(PIN, LOW); time.sleep(0.05)

threading.Thread(target=led_worker, daemon=True).start()

# Caller (no thread creation):
def flash_led(self, color, duration=0.2, count=1):
    self._led_q.put_nowait((color, duration, count))

```

L - Pre-Allocated HDMI Frame (PYNQ)

Call `hdmi_out.newframe()` once at startup and reuse the same buffer every frame. Eliminates per-frame numpy allocation and reduces GC pressure on the PYNQ board's limited RAM (512 MB on Z1).

```

# At startup (once):
self._hdmi_frame = self._hdmi_out.newframe()

# Each frame (zero allocation):
self._hdmi_frame[:copy_h, :copy_w, :] = display[:copy_h, :copy_w, :]
self._hdmi_out.writeframe(self._hdmi_frame)

```

3.3 Optimisation Summary Table

ID	Strategy	Status	Expected Gain
A	Backend priority chain (MediaPipe first)	Applied	Maximises FPS on any platform
B	Skip capture & detect when paused	Applied	0% CPU during pause
C	Sticky bool flag for round scoring	Applied	O(1) per frame vs O(faces)
D	Direct HDMI write on PYNQ	Applied	Removes X11 overhead
E	Mirror via NumPy slice on PYNQ	Applied	Avoids full-frame copy
F	Daemon threads for I/O-bound work	Applied	Zero main-loop blocking
G	Camera buffer size = 1	Proposed	Reduces input latency by 3-5 frames
H	Downscale input before detection	Proposed	+10-20% FPS, same accuracy
I	Detection frame skipping (N=2)	Proposed	+90% throughput, no game impact
J	Producer-consumer detection thread	Proposed	Decouples camera and detector FPS
K	Persistent LED worker thread	Proposed	Removes thread-spawn jitter
L	Pre-allocate PYNQ HDMI frame buffer	Proposed	Eliminates 30+ allocs/sec (PYNQ)